

K8s 安装部署文档



北京首钢自动化信息技术有限公司

2024 年 4 月



版 本 号	创 建 时 间	作 者	受控状态	审核一	审核二
1.0	2024-04-03	朱泽			

文件修改记录

序号	修改后版本号	修改日期	修改人	修改原因/说明	修改内容

目 录

1	k8s 集群 Master 节点服务器 3 台	4
1.1	操作环境	4
1.2	主机信息检查	4
1.2.1	网络相关检查	4
1.2.2	CPU、内存检查	6
1.2.3	目录相关检查	7
1.2.4	修改/etc/hosts	7
1.3	安装 KEEPALIVE+HAPOXY	8
1.3.1	安装 <i>keepalived</i>	8
1.3.2	安装 <i>haproxy</i>	10
1.4	安装 DOCKER	11
1.5	安装 MASTER 节点	11
1.5.1	安装 k8s 组件	12
1.5.2	修改 kubelet 的 cgroup drive	12
1.5.3	初始化集群	12
1.5.4	网络配置	14
1.5.5	其他 master 节点加入集群	15
1.6	回退历史版本	16
2	k8s 集群 node 节点服务器 9 台	17
2.1	操作环境	18
2.2	主机信息检查	18
2.2.1	网络相关检查	18
2.2.2	磁盘相关检查	20
2.2.3	CPU、内存检查	20
2.2.4	目录相关检查	21
2.2.5	修改/etc/hosts	21
2.3	安装 DOCKER	22
2.4	安装 k8s 组件	22
2.5	修改 KUBELET 的 CGROUP DRIVE	23

1 k8s 集群 Master 节点服务器 3 台

序号	主机名	地址	应用包部署位置	部署基础软件	备注
1	Master-01	192.168.4.40/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
2	Master-02	192.168.4.41/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
3	Master-03	192.168.4.42/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
		192.168.4.43/24			浮动 IP

1.1操作环境

CentOS Linux release 7.7.1908 (Core)

1.2主机信息信息检查

1.2.1 网络相关检查

- 1) 检查确认主机名，与规划文档进行核对确认主机名是否需要修改。

检查：

```
[root@localhost ~]# hostnamectl
Static hostname: localhost.localdomain
Icon name: computer-vm
Chassis: vm
Machine ID: 8633a1bd91e447908a40c0df8eaec169
Boot ID: f293dbd6c7d642439394156c7cffe519
Virtualization: vmware
Operating System: CentOS Linux 7 (Core)
CPE OS Name: cpe:/o:centos:centos:7
Kernel: Linux 3.10.0-514.el7.x86_64
Architecture: x86-64
```

修改:

```
[root@localhost /]# hostnamectl --static set-hostname master-01  
[root@sitpp1 ~]# hostnamectl
```

```
Static hostname: master-01  
Icon name: computer-vm  
Chassis: vm  
Machine ID: e862723e199f4a488f6022d0a81fb586  
Boot ID: 49314ccefa7c452596bdf9b6a8d6bf6a  
Virtualization: vmware  
Operating System: CentOS Linux 7 (Core)  
CPE OS Name: cpe:/o:centos:centos:7  
Kernel: Linux 3.10.0-693.el7.x86_64  
Architecture: x86-64
```

2) 执行以下命令确认是否删除虚拟网络。

检查方法如下:

```
[root@localhost /]# ip addr list  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000  
    link/ether 52:54:00:1c:74:7f brd ff:ff:ff:ff:ff:ff  
    inet 192.168.4.40/24 brd 192.168.2.255 scope global noprefixroute dynamic eth0  
        valid_lft 72673sec preferred_lft 72673sec  
    inet6 fe80::5054:ccff:fe1c:747f/64 scope link  
        valid_lft forever preferred_lft forever
```

3) 时钟同步

查看时区

```
# timedatectl  
  
Local time: Thu 2020-05-21 08:19:02 CST  
Universal time: Thu 2020-05-21 00:19:02 UTC  
RTC time: Thu 2020-05-21 00:19:09  
Time zone: Asia/Shanghai (CST, +0800)  
NTP enabled: yes  
NTP synchronized: yes  
RTC in local TZ: no  
DST active: n/a
```

4) 设置时区

```
# timedatectl set-timezone Asia/Shanghai
```

5) 配置时间同步

chrony 方式配置

```
vim /etc/chrony.conf
#添加以下内容
server 192.168.147.80 iburst
server ntp1.aliyun.com iburst
[root@localhost]# systemctl start chronyd.service
[root@localhost]# systemctl restart chronyd.service
[root@localhost]# systemctl status chronyd.service
[root@localhost]# systemctl enable chronyd.service
[root@localhost]# chronyc sources -v
[root@localhost]# chronyc sourcestats -v
```

6) SELINUX

```
[root@localhost /]# sed -i 's/SELINUX=enforcing/SELINUX=disabled/' /etc/selinux/config
[root@localhost /]# grep SELINUX=disabled /etc/selinux/config
SELINUX=disabled
[root@localhost /]# setenforce 0
```

7) 防火墙设置

firewall 关闭

```
[root@localhost /]# systemctl stop firewalld.service
[root@localhost /]# systemctl disable firewalld.service
```

Swap 关闭

```
[root@localhost /]# swapoff -a
[root@localhost /]# vim /etc/fstab      注释 LABEL=YUNIFYSWAP swap
```

1.2.2 CPU、内存检查

确认 CPU、内存与规划大小是否一致，确认分区是否一致

查看 CPU 个数

```
# cat /proc/cpuinfo | grep "physical id" | uniq | wc -l
[root@master-01 ~]# cat /proc/cpuinfo | grep "physical id" | uniq | wc -l
4
```

****uniq 命令：删除重复行;wc -l 命令：统计行数****

查看 CPU 核数

```
# cat /proc/cpuinfo | grep "cpu cores" | uniq
[root@master-01 ~]# cat /proc/cpuinfo | grep "cpu cores" | uniq
cpu cores      : 1
```

查看 CPU 型号

```
# cat /proc/cpuinfo | grep 'model name' | uniq
[root@master-01 ~]# cat /proc/cpuinfo | grep 'model name' | uniq
model name      : QEMU Virtual CPU
```

查看内存总数

```
#cat /proc/meminfo | grep MemTotal
[root@master-01 ~]# cat /proc/meminfo | grep MemTotal
MemTotal:      8008696 kB #8GB
```

1.2.3 目录相关检查

检查介质目录

```
[root@master-01 /]# ll /software
```

创建介质目录

```
[root@master-01 /]# mkdir /software
```

将相关介质放置该目录备用

1.2.4 修改/etc/hosts

```
cat >> /etc/hosts << EOF
192.168.4.40 master-01
192.168.4.41 master-02
192.168.4.42 master-03
192.168.4.43 master-vip
192.168.4.11 node-01
192.168.4.12 node-02
192.168.4.13 node-03
192.168.4.14 node-04
192.168.4.15 node-05
192.168.4.16 node-06
192.168.4.17 node-07
192.168.4.18 node-08
192.168.4.19 node-09
192.168.4.20 node-10
192.168.4.21 node-11
192.168.4.22 node-12
192.168.4.37 cron-01
192.168.4.38 cron-02
192.168.4.57 monitor-01
192.168.4.58 monitor-02
```

EOF

1.3 安装 keepalive+haproxy

1.3.1 安装 keepalived

Yum 安装 keepalived

```
yum install-y keepalived
```

修改 Keepalived 配置

Master-01 节点配置

```
[root@master-01 ~]# vim /etc/keepalived/keepalived.conf
```

```
vrrp_script haproxy-check {  
    script "killall -0 haproxy"  
    interval 2  
    weight 20  
}
```

```
vrrp_instance haproxy-vip {  
    state BACKUP  
    priority 101  
    interface eth0  
    virtual_router_id 47  
    advert_int 3  
    unicast_src_ip 192.168.4.40  
    unicast_peer {  
        192.168.4.41  
        192.168.4.42  
    }  
    virtual_ipaddress {  
        192.168.4.43  
    }  
    track_script {  
        haproxy-check weight 20  
    }  
}
```

Master-02 节点配置

```
[root@master-02 ~]# vim /etc/keepalived/keepalived.conf
```

```
vrrp_script haproxy-check {  
    script "killall -0 haproxy"  
    interval 2  
    weight 20
```



```
}  
vrrp_instance haproxy-vip {  
    state BACKUP  
    priority 101  
    interface eth0  
    virtual_router_id 47  
    advert_int 3  
    unicast_src_ip 192.168.2.41  
    unicast_peer {  
        192.168.2.40  
        192.168.2.42  
    }  
    virtual_ipaddress {  
        192.168.2.43  
    }  
    track_script {  
        haproxy-check weight 20  
    }  
}
```

Master-03 节点配置

[root@master-01 ~]# vim /etc/keepalived/keepalived.conf

```
vrrp_script haproxy-check {  
    script "killall -0 haproxy"  
    interval 2  
    weight 20  
}  
vrrp_instance haproxy-vip {  
    state BACKUP  
    priority 101  
    interface eth0  
    virtual_router_id 47  
    advert_int 3  
    unicast_src_ip 192.168.2.42  
    unicast_peer {  
        192.168.2.40  
        192.168.2.41  
    }  
    virtual_ipaddress {  
        192.168.2.43  
    }  
    track_script {  
        haproxy-check weight 20  
    }  
}
```

```
}  
}
```

1.3.2 安装 haproxy

yum 安装 haproxy

```
yum install -y haproxy
```

修改 haproxy 配置 (3 台 master 节点配置相同)

```
vim /etc/haproxy/haproxy.cfg  
frontend k8s-api  
    bind 0.0.0.0:8443  
    mode tcp  
    option tcplog  
    default_backend k8s-api  
  
backend k8s-api  
    mode tcp  
    option tcplog  
    option tcp-check  
    balance roundrobin  
    default-server inter 10s downinter 5s rise 2 fall 2 slowstart 60s maxconn 250 maxqueue  
256 weight 100  
    server k8s-api-1 192.168.4.40:6443 check  
    server k8s-api-2 192.168.4.41:6443 check  
    server k8s-api-3 192.168.4.42:6443 check  
  
cat > /etc/haproxy/haproxy.cfg << EOF  
global  
    chroot /var/lib/haproxy  
    daemon  
    group haproxy  
    user haproxy  
    log 127.0.0.1:514 local0 warning  
    pidfile /var/lib/haproxy.pid  
    maxconn 20000  
    spread-checks 3  
    nbproc 8  
    defaults  
    log global  
    mode tcp  
    retries 3
```

```
option redispatch
listen https-apiserver
bind 0.0.0.0:8443 # 指定绑定的端口, ip 都设置为 0.0.0.0, 我这里使用 8443 端口
mode tcp
balance roundrobin
timeout server 15s
timeout connect 15s
server apiserver1 192.168.4.40:6443 check port 6443 inter 5000 fall 5 #转发到
k8s-master1 的 apiserver 上, apiserver 端口默认是 6443
server apiserver2 192.168.4.41:6443 check port 6443 inter 5000 fall 5 #转发到
k8s-master2 的 apiserver 上, apiserver 端口默认是 6443
server apiserver3 192.168.4.42:6443 check port 6443 inter 5000 fall 5 #转发到
k8s-master3 的 apiserver 上, apiserver 端口默认是 6443

EOF
```

1.4 安装 docker

添加 yum 源:

```
yum -y install yum-utils
wget -O /etc/yum.repos.d/epel.repo http://mirrors.aliyun.com/repo/epel-7.repo
yum-config-manager --add-repo
http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
yum makecache
```

安装 docker:

```
yum install docker-ce-cli-18.09.9-3.el7 docker-ce-18.09.9-3.el7 -y
```

查看版本:

```
docker version
```

启动并激活开机启动:

```
systemctl start docker
systemctl enable docker
systemctl status docker
```

1.5 安装 master 节点

<https://www.cnblogs.com/zhengchunyan/p/12612475.html> (master 集群高可用)

1.5.1 安装 k8s 组件

配置 k8s 的 yum 源(阿里仓库):

```
cat <<EOF > /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=http://mirrors.aliyun.com/kubernetes/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=0
repo_gpgcheck=0
gpgkey=http://mirrors.aliyun.com/kubernetes/yum/doc/yum-key.gpg
http://mirrors.aliyun.com/kubernetes/yum/doc/rpm-package-key.gpg
EOF
```

配置内核参数, 将桥接的 IPv4 流量传递到 iptables 的链

```
cat > /etc/sysctl.d/k8s.conf << EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
```

安装 k8s 组件:

```
yum install -y kubect1-1.16.2 kubelet-1.16.2 kubeadm-1.16.2
systemctl enable kubelet
systemctl restart kubelet
```

1.5.2 修改 kubelet 的 cgroup drive

查看 docker 的 cgroup drive:

```
[root@master-02 software]# docker info | grep -i cgroup
Cgroup Driver: cgroupfs
```

修改 kubelet 的 cgroup drive:

```
vim /etc/sysconfig/kubelet 改为下面的配置
KUBELET_EXTRA_ARGS=--cgroup-driver=cgroupfs
```

1.5.3 初始化集群

```
kubeadm init --apiserver-advertise-address=192.168.83.129 --image-repository
registry.aliyuncs.com/google_containers --kubernetes-version v1.16.2 --service-cidr=10.1.0.0/16
--pod-network-cidr=10.244.0.0/16 --control-plane-endpoint master-vip:8443
```

```
# --pod-network-cidr= 指定集群使用的网段,默认使用 10.244.0.0/16
# --kubernetes-version= k8s 的版本号
# --service-cidr= svc 网络
# --apiserver-advertise-address= k8s 的 apiserver 地址(启动 master 的服务器 IP 地址)
# 执行 kubeadm init 出错或强制终止,再执行该命令时,需要先执行 kubeadm reset 重置
#使用 kubeadm token create --print-join-command 方法重新生成加入集群的命令
```

成功显示:

```
You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/

You can now join any number of control-plane nodes by copying certificate authorities
and service account keys on each node and then running the following as root:

kubeadm join 192.168.2.40:6443 --token 5k0l0d.h7ntash6n81azqq1 \
--discovery-token-ca-cert-hash
sha256:b490ec43c143a44b9866b873fefe015fe9340d881467eb0e4b471d7d69c6bb09 \
--control-plane

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.2.40:6443 --token 5k0l0d.h7ntash6n81azqq1 \
--discovery-token-ca-cert-hash
sha256:b490ec43c143a44b9866b873fefe015fe9340d881467eb0e4b471d7d69c6bb09
```

```
<none>
[root@master ~]# kubectl get po -A -o wide
NAMESPACE   NAME             READY   STATUS    RESTARTS   AGE   IP              NODE   NOMINATED
NODE   READINESS GATES
kube-system coredns-58cc8c89f4-8tccw 0/1     Pending   0          8m13s <none>          <none> <none>
<none>
kube-system coredns-58cc8c89f4-hrptj 0/1     Pending   0          8m13s <none>          <none> <none>
<none>
```

##coredns 组件如果启动为 pending 状态需要安装 flannel 服务后恢复正常

#master 初始化后生成的命令节点加入集群用

```
kubeadm join 192.168.2.40:6443 --token 5k0l0d.h7ntash6n81azqq1 \
--discovery-token-ca-cert-hash
sha256:b490ec43c143a44b9866b873fefe015fe9340d881467eb0e4b471d7d69c6bb09 \
--control-plane

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.2.40:6443 --token 5k0l0d.h7ntash6n81azqq1 \
--discovery-token-ca-cert-hash
sha256:b490ec43c143a44b9866b873fefe015fe9340d881467eb0e4b471d7d69c6bb09
```

#带有 --control-plane 的用于其他 master 节点加入集群使用

#要等初始化的 master 节点启动 flannel 服务后,其他 master 节点才能加入集群

配置常规用户如何使用 kubectl 访问集群: #Master 所有节点都执行

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

#根据初始化完成时提供的信息,执行以下命令

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

查看 k8s 版本号:

```
[root@localhost ~]# kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"16", GitVersion:"v1.16.2",
GitCommit:"c97fe5036ef3df2967d086711e6c0c405941e14b", GitTreeState:"clean",
BuildDate:"2019-10-15T19:15:39Z", GoVersion:"go1.12.10", Compiler:"gc",
Platform:"linux/amd64"}
```

初始化一个 master 后把生成的 ssh 证书传到另外两个节点

```
#在其他 master 节点创建相应目录
mkdir -p /etc/kubernetes/pki
mkdir -p /etc/kubernetes/pki/etcd
传送 ssh 证书
scp -r /etc/kubernetes/admin.conf master-02:/etc/kubernetes/
scp -r /etc/kubernetes/pki/ca* master-02:/etc/kubernetes/pki/
scp -r /etc/kubernetes/pki/sa* master-02:/etc/kubernetes/pki/
scp -r /etc/kubernetes/pki/front* master-02:/etc/kubernetes/pki/
scp -r /etc/kubernetes/pki/etcd/ca* master-02:/etc/kubernetes/pki/etcd/

scp -r /etc/kubernetes/admin.conf master-03:/etc/kubernetes/
scp -r /etc/kubernetes/pki/ca* master-03:/etc/kubernetes/pki/
scp -r /etc/kubernetes/pki/sa* master-03:/etc/kubernetes/pki/
scp -r /etc/kubernetes/pki/front* master-03:/etc/kubernetes/pki/
scp -r /etc/kubernetes/pki/etcd/ca* master-03:/etc/kubernetes/pki/etcd/
```

1.5.4 网络配置



flannel.yaml

##创建 kube-flannel.yaml 文件，添加以下内容 (这一步不需要做)

```
apiVersion: kubeadm.k8s.io/v1beta1
kind: ClusterConfiguration
kubernetesVersion: v1.16.2

image-repository:
  registry.aliyuncs.com/google_containers

apiServerCertSANS:
- 172.16.1.6
- 172.16.1.5
```

```
- 172.16.1.4
- master-01
- master-02
- master-03
- 172.16.1.200
api:
  controlPlaneEndpoint: 172.16.1.200:8443
etcd:
  external:
    endpoints:
      - https://172.16.1.6:2379
      - https://172.16.1.5:2379
      - https://172.16.1.4:2379
    caFile: /etc/etcd/ssl/ca.pem
    certFile: /etc/etcd/ssl/etcd.pem
    keyFile: /etc/etcd/ssl/etcd-key.pem
networking:
  # This CIDR is a Calico default. Substitute or remove for your CNI provider.
```

对网络资源进行配置（在初始化的 master 节点执行）

```
kubect1 apply -f flannel.yml
```

1.5.5 其他 master 节点加入集群

```
kubeadm join master-vip:8443 --token 7div0n.j29wxkqhzbvgqxxm \
--discovery-token-ca-cert-hash
sha256:9eb69e437335eedf3b86510fe0205f0e40c5508eeb3abc1b99dd290c96da96a8 \
--control-plane
```

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join master-vip:8443 --token 1mdbyd.m337tn22fdb15k5g
--discovery-token-ca-cert-hash
sha256:9eb69e437335eedf3b86510fe0205f0e40c5508eeb3abc1b99dd290c96da96a8
```

```
kubeadm join 192.168.2.40:6443 --token 5k010d.h7ntash6n81azqq1 \
--discovery-token-ca-cert-hash
sha256:b490ec43c143a44b9866b873fefe015fe9340d881467eb0e4b471d7d69c6bb09 \
--control-plane
```

#配置 secret 在 master 节点执行

在 k8s 中拉取私有镜像仓库的镜像，需要配置 secret:

```
###kubect1 create secret docker-registry registry-secret-name --docker-server=192.168.2.46:5000
--docker-username=admin --docker-password=Harhor123456 -n namespace
```

#创建

kubectl create namespace app

kubectl create secret docker-registry harbor --docker-server=10.68.27.22:5000

--docker-username=admin --docker-password=Harhor123456 -n app

#需要指定

#1.registry-secret-name 名称

#2.namespace 命名空间

#3. --docker-server 仓库地址加端口号，harbor 改了端口号，此处加端口号，默认 80 不用加

#4.--docker-username --docker-password 用户名密码（提前新建好）

#查看

kubectl get secret -n namespace

创建 kubectl create -f deployment.yaml 文件需要在 spec.template.spec 中增加 imagePullSecrets，name 值为刚刚创建的 secret 名称：harbor

```
12     app: accessmanage-center
13   template:
14     metadata:
15       labels:
16         app: accessmanage-center
17     spec:
18       imagePullSecrets:
19         - name: registry-secret-name
20     containers:
21       - name: accessmanage-center
22         image: 10.244.1.3:5000/tomcat:7
```

1.6回退历史版本

#查看版本包 kubectl rollout history deployment ccs-web-vue-deployment -n prod

```
[root@packing-01 ccs-web-vue]# kubectl rollout history deployment ccs-web-vue-deployment -n prod
deployment.apps/ccs-web-vue-deployment
REVISION  CHANGE-CAUSE
89        <none>
91        <none>
92        <none>
93        <none>
94        <none>
95        <none>
96        <none>
97        <none>
100       <none>
101       <none>
102       <none>
```

#回退版本，最下面的 revision 是最新版本

kubectl rollout undo deployment ccs-web-vue-deployment --to-revision=98 -n prod

2 k8s 集群 node 节点服务器 9 台

序号	主机名	地址	应用包部署位置	部署基础软件	备注
1	node-01	192.168.2.11/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
2	node-02	192.168.2.12/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
3	node-03	192.168.2.13/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
4	node-04	192.168.2.14/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
5	node-05	192.168.2.15/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
6	node-06	192.168.2.16/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
7	node-07	192.168.2.17/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
8	node-07	192.168.2.18/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
9	node-09	192.168.2.19/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
10	node-10	192.168.2.20/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
11	node-11	192.168.2.21/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	
12	node-12	192.168.2.22/24	/usr/bin	kubelet-1.16.2 kubeadm-1.16.2 kubect1-1.16.2	

2.1 操作环境

CentOS Linux release 7.7.1908 (Core)

2.2 主机信息检查

2.2.1 网络相关检查

8) 检查确认主机名，与规划文档进行核对确认主机名是否需要修改。

检查：

```
[root@localhost ~]# hostnamectl
Static hostname: localhost.localdomain
Icon name: computer-vm
Chassis: vm
Machine ID: 8633a1bd91e447908a40c0df8eaec169
Boot ID: f293dbd6c7d642439394156c7cffe519
Virtualization: vmware
Operating System: CentOS Linux 7 (Core)
CPE OS Name: cpe:/o:centos:centos:7
Kernel: Linux 3.10.0-514.el7.x86_64
Architecture: x86-64
```

修改：

```
[root@localhost ~]# hostnamectl --static set-hostname node-01
[root@sitpp1 ~]# hostnamectl
Static hostname: node-01
Icon name: computer-vm
Chassis: vm
Machine ID: e862723e199f4a488f6022d0a81fb586
Boot ID: 49314ccefa7c452596bdf9b6a8d6bf6a
Virtualization: vmware
Operating System: CentOS Linux 7 (Core)
CPE OS Name: cpe:/o:centos:centos:7
Kernel: Linux 3.10.0-693.el7.x86_64
Architecture: x86-64
```

9) 执行以下命令确认是否删除虚拟网络。

检查方法如下：

```
[root@localhost ~]# ip addr list
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default
```

```
qlen 1000
link/ether 52:54:cc:1c:74:7f brd ff:ff:ff:ff:ff:ff
inet 192.168.4.11/24 brd 192.168.2.255 scope global noprefixroute dynamic eth0
    valid_lft 72673sec preferred_lft 72673sec
inet6 fe80::5054:ccff:fe1c:747f/64 scope link
    valid_lft forever preferred_lft forever
```

10) 时钟同步

查看时区

```
# timedatectl

Local time: Thu 2020-05-21 08:19:02 CST
Universal time: Thu 2020-05-21 00:19:02 UTC
RTC time: Thu 2020-05-21 00:19:09
Time zone: Asia/Shanghai (CST, +0800)
NTP enabled: yes
NTP synchronized: yes
RTC in local TZ: no
DST active: n/a
```

设置时区

```
# timedatectl set-timezone Asia/Shanghai
```

配置时间同步

chrony 方式配置

```
vim /etc/chrony.conf
#添加以下内容
server 192.168.147.80 iburst
server ntp1.aliyun.com iburst
[root@localhost]# systemctl start chronyd.service
[root@localhost]# systemctl restart chronyd.service
[root@localhost]# systemctl status chronyd.service
[root@localhost]# systemctl enable chronyd.service
[root@localhost]# chronyc sources -v
[root@localhost]# chronyc sourcestats -v
```

11) SELINUX

```
[root@localhost /]# sed -i 's/SELINUX=enforcing/SELINUX=disabled/' /etc/selinux/config
[root@localhost /]# grep SELINUX=disabled /etc/selinux/config
SELINUX=disabled
[root@localhost /]# setenforce 0
```

12) 防火墙设置

firewall 关闭

```
[root@localhost /]# systemctl stop firewalld.service
[root@localhost /]# systemctl disable firewalld.service
```

Swap 关闭

```
[root@localhost ~]# swapoff -a
[root@localhost ~]# vim /etc/fstab      注释 LABEL=YUNIFYSWAP swap
```

2.2.2 磁盘相关检查

确认磁盘大小与规划大小是否一致，确认分区是否一致

```
[root@master ~]# fdisk -l

Disk /dev/vda: 85.9 GB, 85899345920 bytes, 167772160 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0x0003d0a9

   Device Boot      Start         End      Blocks   Id  System
   /dev/vda1    *        2048     167770207     83884080   83   Linux

Disk /dev/vdb: 8589 MB, 8589934592 bytes, 16777216 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

2.2.3 CPU、内存检查

确认 CPU、内存与规划大小是否一致，确认分区是否一致

查看 CPU 个数

```
# cat /proc/cpuinfo | grep "physical id" | uniq | wc -l
[root@node-01 ~]# cat /proc/cpuinfo | grep "physical id" | uniq | wc -l
8
```

****uniq 命令：删除重复行;wc -l 命令：统计行数****

查看 CPU 核数

```
# cat /proc/cpuinfo | grep "cpu cores" | uniq
[root@node-01 ~]# cat /proc/cpuinfo | grep "cpu cores" | uniq
cpu cores      : 1
```

查看 CPU 型号

```
# cat /proc/cpuinfo | grep 'model name'| uniq
[root@node-01 ~]# cat /proc/cpuinfo | grep 'model name'| uniq
model name     : QEMU virtual CPU
```

查看内存总数

```
#cat /proc/meminfo | grep MemTotal
```

```
[root@node-01 ~]# cat /proc/meminfo | grep MemTotal  
MemTotal:      16265588 kB    #16GB
```

2.2.4 目录相关检查

检查介质目录

```
[root@node-01 /]# ll /software
```

创建介质目录将相关介质放置该目录备用

```
[root@node-01 /]# mkdir /software
```

2.2.5 修改/etc/hosts

```
cat >> /etc/hosts << EOF  
192.168.4.40 master-01  
192.168.4.41 master-02  
192.168.4.42 master-03  
192.168.4.43 master-vip  
192.168.4.11 node-01  
192.168.4.12 node-02  
192.168.4.13 node-03  
192.168.4.14 node-04  
192.168.4.15 node-05  
192.168.4.16 node-06  
192.168.4.17 node-07  
192.168.4.18 node-08  
192.168.4.19 node-09  
192.168.4.20 node-10  
192.168.4.21 node-11  
192.168.4.22 node-12  
192.168.4.34 capability-01  
192.168.4.35 capability-02  
192.168.4.37 cron-01  
192.168.4.38 cron-02  
192.168.4.57 monitor-01  
192.168.4.58 monitor-02  
  
EOF
```

2.3 安装 docker

所有 node 节点添加 yum 源:

```
yum -y install yum-utils
yum-config-manager --add-repo
http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
yum makecache
```

所有 node 节点安装 docker:

```
yum install docker-ce-cli-18.09.9-3.el7 docker-ce-18.09.9-3.el7 -y
```

查看版本:

```
[root@node-01 ~]# docker version
Client:
Version:      18.09.9
API version:  1.39
Go version:   go1.11.13
Git commit:   039a7df9ba
Built:        wed Sep  4 16:51:21 2019
OS/Arch:      linux/amd64
Experimental: false
```

所有 node 节点启动并激活开机自启动:

```
systemctl start docker
systemctl enable docker
systemctl status docker
```

2.4 安装 k8s 组件

所有 node 节点配置 k8s 的 yum 源(阿里仓库):

```
cat <<EOF > /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=kubernetes
baseurl=http://mirrors.aliyun.com/kubernetes/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=0
repo_gpgcheck=0
gpgkey=http://mirrors.aliyun.com/kubernetes/yum/doc/yum-key.gpg
http://mirrors.aliyun.com/kubernetes/yum/doc/rpm-package-key.gpg
EOF
```

配置内核参数, 将桥接的 IPv4 流量传递到 iptables 的链

```
cat > /etc/sysctl.d/k8s.conf << EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
```

所有 node 节点安装组件:

```
yum install -y kubect1-1.16.2 kubelet-1.16.2 kubeadm-1.16.2  
systemctl enable kubelet
```

2.5 修改 kubelet 的 cgroup drive

所有 node 节点查看 docker 的 cgroup drive:

```
[root@node-01 software]# docker info | grep -i cgroup  
Cgroup Driver: cgroupfs
```

所有 node 节点修改 kubelet 的 cgroup drive:

```
vim /etc/sysconfig/kubelet 改为下面的配置  
KUBELET_EXTRA_ARGS=--cgroup-driver=cgroupfs
```

加入集群

```
kubeadm join master-vip:8443 --token 1mdbyd.m337tn22fdb15k5g  
--discovery-token-ca-cert-hash  
sha256:9eb69e437335eedf3b86510fe0205f0e40c5508eeb3abc1b99dd290c96da96a8
```